

# Function

functions are known as methods and must always reside inside a class

## Components:

- Return type
- Method type
- Parameters
- Method body

## Types of functions

1. Based on parameter and return type
2. Based on where they are declared
3. Based on functionality

### Type 1:

- No parameter & no return- Methods doesn't take & doesn't return output
- Parameter but no return value-take input and doesn't return output
- No parameter but return value –Doesn't take input but return a value
- Parameter and return value-Takes input and return the result

### Type 2:

- Instance method-belongs to an object of the class.
- Static Method-belongs to class itself.
- Abstract Method-declared without implementation and defined in subclass
- Final method-Cannot be overwritten by subclass
- Native Method-written in another language (like c,c++) linked to java.

### Type 3:

- User defined method-Created by the programmer
- Pre-defined method-Already available in java library

## Constructor

special block of code similar to a method that is automatically invoked when an instance (object) of a class is created. Its primary purpose is to initialize the object's instance variables and allocate memory.

## Types;

### 1. Default Constructor (No-Argument)

If you do not define any constructor in your class, the Java compiler automatically inserts an empty, no-argument constructor at compile-time. It initializes object variables with their standard default values (e.g., 0 for numbers, null for objects).

#### Syntax:

```
class Car {
    String model; // Defaults to null
    int speed;    // Defaults to 0
}
// Usage: Car c = new Car(); // Calls the compiler-generated default
// constructor
```

### 2. Parameterized Constructor

A constructor explicitly written by the developer that accepts specific arguments. It allows you to assign custom values to an object's variables upon creation.

#### Syntax

```
class Car {
    String model;

    // Parameterized constructor
    Car(String modelName) {
        this.model = modelName;
    }
}
// Usage: Car c = new Car("Tesla");
```

### 3. Copy Constructor

Unlike languages like C++, Java does not provide an automatic copy constructor. However, you can manually write one by passing an object of the same class as a parameter to duplicate data

#### Syntax:

```
class Car {
    String model;

    Car(String model) { this.model = model; }

    // Copy Constructor
```

```

        Car(Car existingCar) {
            this.model = existingCar.model;
        }
    }
    // Usage:
    // Car c1 = new Car("BMW");
    // Car c2 = new Car(c1); // Duplicates c1 into c2

```

## Constructor Overloading

Constructor Overloading is a technique that allows a class to have more than one constructor, provided they have different parameter lists. It is a form of compile-time polymorphism that gives you flexibility to initialize objects in multiple ways depending on the data available

### Example

```

class Product {
    String name;
    double price;
    int stock;

    // Constructor 1: No arguments (sets defaults)
    Product() {
        this.name = "Generic Item";
        this.price = 0.0;
        this.stock = 0;
    }

    // Constructor 2: One argument (Overloaded by number of parameters)
    Product(String name) {
        this.name = name;
        this.price = 9.99; // Default promo price
        this.stock = 1;
    }
}

```

```
// Constructor 3: Three arguments (Overloaded by number of parameters)
Product(String name, double price, int stock) {
    this.name = name;
    this.price = price;
    this.stock = stock;
}

void display() {
    System.out.println(name + " | Price: $" + price + " | Stock: " + stock);
}

}

public class Main {
    public static void main(String[] args) {
        // Triggers Constructor 1
        Product p1 = new Product();

        // Triggers Constructor 2
        Product p2 = new Product("Laptop");

        // Triggers Constructor 3
        Product p3 = new Product("Smartphone", 799.99, 50);

        p1.display(); // Output: Generic Item | Price: $0.0 | Stock: 0
        p2.display(); // Output: Laptop | Price: $9.99 | Stock: 1
        p3.display(); // Output: Smartphone | Price: $799.99 | Stock: 50
    }
}
```

